

Variables and Data Types

Features for declaring and managing variables and data types in PHP.

Features: \$variable, int, float, string, bool, array, object, null, resource

Feature	Uses	Example	Practical Uses
\$variable	Declares a variable with dynamic typing.	<code>\$name = "John";</code>	Storing user input or session data.
int	Integer data type.	<code>\$count = 42;</code>	Counters for loops or IDs.
float	Floating-point number.	<code>\$price = 19.99;</code>	Prices or measurements in forms.
string	Text data type.	<code>\$title = "Welcome";</code>	Displaying text in web pages.
bool	Boolean (true/false) type.	<code>\$isActive = true;</code>	Toggling features or permissions.
array	Ordered or associative collection.	<code>\$items = ['apple', 'banana'];</code>	Storing lists like menu items.
object	Instance of a class.	<code>\$user = new stdClass; \$user->name = "John";</code>	Representing structured data like users.
null	Represents no value.	<code>\$data = null;</code>	Clearing variables or default states.
resource	Reference to external resource (e.g., file, database).	<code>\$file = fopen("data.txt", "r");</code>	Handling files or database connections.

Control Structures

Features for controlling program flow.

Features: if, else, elseif, switch, for, foreach, while, do-while, break, continue

Feature	Uses	Example	Practical Uses
if	Conditional execution.	<code>if (\$age > 18) { echo "Adult"; }</code>	Restricting access by age.
else	Alternative execution path.	<code>if (\$loggedIn) {} else { echo "Please log in"; }</code>	Handling unauthenticated users.
elseif	Additional conditional check.	<code>if (\$score > 80) {} elseif (\$score > 50) { echo "Pass"; }</code>	Grading systems in forms.
switch	Multi-way branching.	<code>switch (\$day) { case 1: echo "Monday"; break; }</code>	Handling form input options.
for	Loops with counter.	<code>for (\$i = 0; \$i < 5; \$i++) { echo \$i; }</code>	Generating table rows.
foreach	Iterates over arrays.	<code>foreach (\$items as \$item) { echo \$item; }</code>	Displaying product lists.
while	Loops while condition is true.	<code>while (\$count > 0) { \$count--; }</code>	Processing queued tasks.
do-while	Loops at least once.	<code>do { echo \$x--; } while (\$x > 0);</code>	Ensuring one iteration for data fetch.
break	Exits loop or switch.	<code>for (\$i = 0; \$i < 5; \$i++) { if (\$i == 3) break; }</code>	Stopping loop on condition.
continue	Skips current loop iteration.	<code>for (\$i = 0; \$i < 5; \$i++) { if (\$i == 2) continue; }</code>	Skipping invalid data in loops.

Functions

Features for defining and using reusable code blocks.

Features: function, return, default parameters, anonymous function, arrow function (PHP 7.4+), variable function

Feature	Uses	Example	Practical Uses
function	Defines a reusable function.	<code>function add(\$a, \$b) { return \$a + \$b; }</code>	Calculating totals in carts.
return	Exits function with a value.	<code>function getName() { return "John"; }</code>	Returning user data to templates.
default parameters	Sets default argument values.	<code>function greet(\$name = "Guest") { return "Hello, \$name"; }</code>	Optional form inputs.
anonymous function	Unnamed function for callbacks.	<code>\$greet = function(\$name) { echo "Hi, \$name"; };</code>	Event handlers in frameworks.
arrow function	Concise anonymous function (PHP 7.4+).	<code>\$add = fn(\$a, \$b) => \$a + \$b;</code>	Short callbacks in <code>array_map</code> .
variable function	Calls function by its name stored in variable.	<code>\$func = 'add'; \$func(2, 3);</code>	Dynamic function calls in plugins.

Arrays

Features for working with arrays in PHP.

Features: `array()`, `count`, `array_push`, `array_pop`, `array_merge`, `array_map`, `array_filter`, `foreach` (already covered)

Feature	Uses	Example	Practical Uses
<code>array()</code>	Creates an array.	<code>\$fruits = array('apple', 'banana');</code>	Storing lists like form options.
<code>count</code>	Returns number of elements.	<code>count(\$fruits);</code>	Checking list size for validation.
<code>array_push</code>	Adds elements to array end.	<code>array_push(\$fruits, 'orange');</code>	Adding items to a cart.
<code>array_pop</code>	Removes last element.	<code>array_pop(\$fruits);</code>	Removing last item in a queue.
<code>array_merge</code>	Combines arrays.	<code>\$combined = array_merge(\$fruits, \$veggies);</code>	Merging data from multiple sources.
<code>array_map</code>	Applies function to each element.	<code>array_map('strtoupper', \$fruits);</code>	Transforming data for display.
<code>array_filter</code>	Filters elements by callback.	<code>array_filter(\$numbers, fn(\$n) => \$n > 0);</code>	Filtering valid form submissions.

String Functions

Built-in functions for manipulating strings.

Features: `strlen`, `strtoupper`, `strtolower`, `substr`, `str_replace`, `trim`, `explode`, `implode`

Feature	Uses	Example	Practical Uses
<code>strlen</code>	Returns string length.	<code>strlen("Hello");</code>	Validating input length in forms.
<code>strtoupper</code>	Converts string to uppercase.	<code>strtoupper("hello");</code>	Formatting titles for display.
<code>strtolower</code>	Converts string to lowercase.	<code>strtolower("HELLO");</code>	Normalizing user input.

Feature	Uses	Example	Practical Uses
substr	Extracts part of a string.	<code>substr("Hello", 0, 3);</code>	Extracting prefixes like usernames.
str_replace	Replaces part of a string.	<code>str_replace("World", "PHP", "Hello World");</code>	Updating text in templates.
trim	Removes whitespace from both ends.	<code>trim(" Hello ");</code>	Cleaning form inputs.
explode	Splits string into array by delimiter.	<code>explode(",", "a,b,c");</code>	Parsing CSV data.
implode	Joins array into string with delimiter.	<code>implode(",", ['a', 'b', 'c']);</code>	Creating CSV strings for export.

Object-Oriented Programming

Features for defining and working with classes and objects.

Features: class, new, \$this, public/private/protected, extends, interface, abstract, static

Feature	Uses	Example	Practical Uses
class	Defines a blueprint for objects.	<code>class User { public \$name; }</code>	Modeling entities like users.
new	Creates an object instance.	<code>\$user = new User();</code>	Instantiating models for data.
\$this	Refers to current object.	<code>\$this->name = "John";</code>	Accessing instance properties.
public	Accessible everywhere.	<code>public \$name;</code>	Exposing properties like user details.
private	Accessible only within class.	<code>private \$password;</code>	Protecting sensitive data.
protected	Accessible in class and subclasses.	<code>protected \$id;</code>	Sharing data with child classes.
extends	Inherits from parent class.	<code>class Admin extends User {}</code>	Reusing code in user hierarchies.
interface	Defines method contracts.	<code>interface Loggable { public function log(); }</code>	Ensuring class behavior in plugins.
abstract	Defines a class that can't be instantiated.	<code>abstract class Model { abstract function save(); }</code>	Base classes for database models.
static	Class-level member.	<code>static \$count = 0;</code>	Tracking shared data like user count.

Database Interaction

Features for interacting with databases (e.g., MySQL via PDO or mysqli).

Features: PDO, mysqli_connect, prepare, execute, query, fetch

Feature	Uses	Example	Practical Uses
PDO	Database access abstraction.	<code>\$pdo = new PDO("mysql:host=localhost;dbname=test", "user", "pass");</code>	Connecting to databases securely.
mysqli_connect	Connects to MySQL database.	<code>\$conn = mysqli_connect("localhost", "user", "pass", "test");</code>	Establishing database connections.

Feature	Uses	Example	Practical Uses
prepare	Prepares SQL statement for execution.	<code>\$stmt = \$pdo->prepare("SELECT * FROM users WHERE id = ?");</code>	Preventing SQL injection in queries.
execute	Executes prepared statement.	<code>\$stmt->execute([1]);</code>	Running parameterized queries.
query	Executes direct SQL query.	<code>\$result = \$pdo->query("SELECT * FROM products");</code>	Fetching data for simple queries.
fetch	Retrieves query results.	<code>\$row = \$stmt->fetch(PDO::FETCH_ASSOC);</code>	Displaying database results in tables.

HTTP and Request Handling

Features for handling web requests and responses.

Features: *\$_GET*, *\$_POST*, *\$_SESSION*, *header*, *\$_SERVER*, *filter_input*

Feature	Uses	Example	Practical Uses
<code>\$_GET</code>	Accesses URL query parameters.	<code>\$id = \$_GET['id'];</code>	Retrieving IDs from URLs.
<code>\$_POST</code>	Accesses form POST data.	<code>\$name = \$_POST['name'];</code>	Processing form submissions.
<code>\$_SESSION</code>	Stores session data.	<code>\$_SESSION['user'] = 'John';</code>	Maintaining user login state.
<code>header</code>	Sets HTTP headers.	<code>header("Location: /home.php");</code>	Redirecting users after login.
<code>\$_SERVER</code>	Accesses server and request info.	<code>\$ip = \$_SERVER['REMOTE_ADDR'];</code>	Logging user IP addresses.
<code>filter_input</code>	Sanitizes/validates input.	<code>\$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);</code>	Validating form inputs.

File Handling

Features for working with files and directories.

Features: *file_get_contents*, *file_put_contents*, *fopen*, *fclose*, *fread*, *fwrite*, *unlink*

Feature	Uses	Example	Practical Uses
<code>file_get_contents</code>	Reads file into string.	<code>\$data = file_get_contents("config.txt");</code>	Loading configuration files.
<code>file_put_contents</code>	Writes string to file.	<code>file_put_contents("log.txt", "Error");</code>	Logging errors to a file.
<code>fopen</code>	Opens a file or URL.	<code>\$file = fopen("data.txt", "r");</code>	Reading CSV files for import.
<code>fclose</code>	Closes an open file.	<code>fclose(\$file);</code>	Releasing file resources.
<code>fread</code>	Reads from an open file.	<code>\$content = fread(\$file, filesize("data.txt"));</code>	Reading file contents for display.
<code>fwrite</code>	Writes to an open file.	<code>fwrite(\$file, "Hello");</code>	Writing user data to files.
<code>unlink</code>	Deletes a file.	<code>unlink("temp.txt");</code>	Removing temporary files.

Error and Exception Handling

Features for managing errors and exceptions.

Features: try-catch, throw, set_error_handler, error_reporting

Feature	Uses	Example	Practical Uses
try-catch	Handles exceptions.	<pre>try { \$db->query("INVALID"); } catch (Exception \$e) { echo \$e->getMessage(); }</pre>	Handling database errors.
throw	Throws an exception.	<pre>throw new Exception("Invalid input");</pre>	Custom error messages in forms.
set_error_handler	Sets custom error handler.	<pre>set_error_handler(function(\$errno, \$errstr) { log(\$errstr); });</pre>	Logging runtime errors.
error_reporting	Configures error reporting level.	<pre>error_reporting(E_ALL);</pre>	Debugging during development.